Build an MCP Client

Estimated time needed: 30 minutes

In this lab, you will build an MCP client that connects to the MCP server, discovers its tools, and invokes them. You will also implement roots and sampling so the client can properly participate in the MCP protocol.

Learning objectives

By the end of this lab, you will have:

- Connected to a running MCP server using `ClientSession` over stdio
- Implemented a **roots callback** to tell the server which directories it can access
- Implemented a **sampling callback** to handle LLM requests delegated from the server
- Called all three server tools through the MCP protocol and inspected their output

Set up the environment

First, create a virtual environment to keep dependencies isolated.

bash

```
pip install virtualenv
virtualenv .venv
source .venv/bin/activate
```

Run

Install the required libraries:

bash

```
pip install fastmcp==3.1.0 anthropic==0.84.0 mcp==1.25.0
```

Run

Download the data files and the server from the previous lab:

bash

```
# Unstructured California culinary map
curl -O https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/R1

# Structured restaurant data (JSON)
curl -O https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/7I

# Augmented user reviews (JSON)
curl -O https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/su

# MCP server from the previous lab
curl -O https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/RL
```

Run

Create the client file:

bash

```
touch client.py
```

Run

Open client.py in IDE

MCP client foundation

Imports and configuration

At the top of `client.py`, import the required libraries and set up the two configuration values the client needs.

python

```python
# Libraries for MCP client, LLM handling, and async operations
import asyncio
import json
from pathlib import Path
from anthropic import Anthropic
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
from mcp.types import (
    Root,
    TextContent,
    CreateMessageResult,
    CreateMessageRequestParams,
)

# Configuration
SERVER_SCRIPT = str(Path(__file__).parent / "server.py")
PROJECT_DIR = Path(__file__).parent.resolve()
```

`SERVER_SCRIPT` is the path to `server.py`—the MCP client will launch it as a subprocess. `PROJECT_DIR` resolves to whichever directory `client.py` lives in, so both values stay correct regardless of where the project is stored.

Launching the server process

Next, define the parameters that tell the MCP library how to start the server.

```python
# StdioServerParameters launches "python server.py" via stdin/stdout
server_params = StdioServerParameters(
    command="python",
    args=[SERVER_SCRIPT],
)
```

`StdioServerParameters` describes the subprocess command. When a `ClientSession` is opened later, the MCP library uses these parameters to start `server.py` and connect to it over stdin/stdout—no network port required.

Roots and sampling

We are still working on `client.py`, so continue to add all the following code to it.

MCP defines two optional client-side callbacks that give the server richer capabilities: **roots** and **sampling**. We will implement both.

Roots: Declaring filesystem access

A **root** is a directory URI that the client declares it is willing to share with the server. The server can request this list to understand what paths it is allowed to read.

python

```python
# ROOTS — Tell the server which directories it can access
def list_roots() -> list[Root]:
    """Limit the server's file access to this project directory."""
    return [Root(uri=f"file://{PROJECT_DIR}", name=PROJECT_DIR.name)]
```

Returning a single `Root` pointing at `PROJECT_DIR` means the server is permitted to read files within the project folder but nothing outside it. `PROJECT_DIR.name` is used as the display name, so it reflects the actual directory name automatically.

Sampling: Delegating LLM calls to the client

Sampling lets the server ask the client to run an LLM call on its behalf. This keeps API keys and model configuration entirely on the client side; the server only sends a prompt and receives a text response.

First, create the Anthropic client that will perform the actual model calls:

```python
# Anthropic client used to fulfill sampling requests from the server
anthropic_client = Anthropic()
```

Then, implement the sampling callback:

```python
# SAMPLING — Handle LLM requests delegated from the server
async def handle_sampling(params: CreateMessageRequestParams) -> CreateMessag
    """Run a Claude LLM call on behalf of the server and return the result.""
    # Extract the prompt text from the first sampling message
    prompt = params.messages[0].content.text

    print(f"\n[Sampling] Server requested LLM task:")
    print(f"  Prompt preview: {prompt[:150]}...")

    response = anthropic_client.messages.create(
        model="claude-sonnet-4-20250514",
        max_tokens=params.maxTokens or 200,
        messages=[{"role": "user", "content": prompt}],
    )

    response_text = response.content[0].text
    print(f"  LLM Response: {response_text[:100]}...")

    return CreateMessageResult(
        role="assistant",
        content=TextContent(type="text", text=response_text),
        model="claude-sonnet-4-20250514",
    )
```

The callback receives a `CreateMessageRequestParams` object containing the server's prompt, calls Claude with it, and wraps the response in `CreateMessageResult` that the MCP library returns to the server. `params.maxTokens` is used when provided, so the server can control response length; a sensible default of `200` is used as a fallback.

Session helper and tool calls

Core session helper

All communication with the server happens inside a `ClientSession`. Rather than repeating the connection boilerplate in every function, we create a single reusable helper.

python

```python
# HELPER — Open a session, call a tool, and return the parsed JSON result
async def call_tool(tool_name: str, arguments: dict) -> dict:
    """Connect to the server, call a tool, and return the parsed JSON result.
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(
            read,
            write,
            sampling_callback=handle_sampling,
            list_roots_callback=list_roots,
        ) as session:
            await session.initialize()
            result = await session.call_tool(tool_name, arguments=arguments)
            # The server always returns a single TextContent item
            return json.loads(result.content[0].text)
```

`stdio_client` starts the server subprocess and provides the `read`/`write` streams. `ClientSession` wraps those streams into the full MCP protocol, registering our two callbacks so the server can trigger them. After `session.initialize()` completes the MCP handshake, `session.call_tool()` sends a JSON-RPC `tools/call` request to the server and returns the raw result, which we parse from JSON before returning.

Connection verification

Verifying the connection

Before running demos, it is good practice to verify that the server starts correctly and exposes the tools and resources we expect.

python

```python
# CONNECTION & DISCOVERY
async def verify_connection():
    """Connect to the server and verify all expected tools and resources exis
    print("=" * 60)
    print("MCP Connection Verification")
    print("=" * 60)

    async with stdio_client(server_params) as (read, write):
        async with ClientSession(
            read,
            write,
            sampling_callback=handle_sampling,
            list_roots_callback=list_roots,
        ) as session:
            await session.initialize()

            # list_tools() sends a "tools/list" JSON-RPC request to the serve
            tools_result = await session.list_tools()
            tool_names = [tool.name for tool in tools_result.tools]
            print("--- START SCREENSHOT ---")
            print(f"\nDiscovered {len(tool_names)} tools:")
            for tool in tools_result.tools:
                print(f"  - {tool.name}: {tool.description[:80]}...")

            assert "get_restaurant_info" in tool_names, "FAIL: get_restaurant
            assert "recommend_by_vibe" in tool_names, "FAIL: recommend_by_vib
            assert "get_review" in tool_names, "FAIL: get_review not found!"
            print("\nAll required tools verified!")

            # list_resources() discovers data endpoints the server exposes
            resources_result = await session.list_resources()
            print(f"\nDiscovered {len(resources_result.resources)} resources:
            for resource in resources_result.resources:
                print(f"  - {resource.uri}: {resource.name}")

            roots = list_roots()
            print(f"\nConfigured {len(roots)} roots:")
            for root in roots:
                print(f"  - {root.name}: {root.uri}")

            print("--- END SCREENSHOT ---")
```

`session.list_tools()` and `session.list_resources()` send standard MCP discovery

any expected tool, the error message will tell you exactly which one.

Demo functions

Demo 1: Get restaurant info

This demo calls the `get_restaurant_info` tool to look up a restaurant by name.

```python
# DEMOS — Call each tool through the MCP protocol
async def demo_get_restaurant_info():
    """Demo: Look up a restaurant by name."""
    print("\n" + "-" * 60)
    print("Demo: get_restaurant_info('Iron & Embers')")
    print("-" * 60)

    data = await call_tool("get_restaurant_info", {"restaurant_name": "Iron &
    print(json.dumps(data, indent=2))
```

The full JSON result—including cuisine, rating, price range, and signature dish—is printed directly. The tool returns a `status` field of `"found"` or `"not_found"` so callers can branch on the result.

Demo 2: Recommend by vibe

This demo calls `recommend_by_vibe` and shows how to navigate its two-section response.

```python
async def demo_recommend_by_vibe():
    """Demo: Find restaurants by vibe keyword."""
    print("\n" + "-" * 60)
    print("Demo: recommend_by_vibe('moody')")
    print("-" * 60)

    data = await call_tool("recommend_by_vibe", {"vibe": "moody"})
    print(f"Vibe: {data['vibe_searched']}")
    print(f"Structured matches: {len(data['structured_matches'])}")
    for match in data["structured_matches"]:
        print(f"  - {match['name']} ({match['cuisine']}) - {match['rating']}/
    print(f"Raw text excerpts: {len(data['raw_text_excerpts'])}")
```

The response contains `structured_matches` (precise hits from the JSON vibe tags) and `raw_text_excerpts` (paragraph-level hits from the culinary map text). Displaying the count of each section gives a quick sense of how well the keyword matched.

Demo 3: Get review

This demo retrieves the full review record for a named restaurant.

```python
async def demo_get_review():
    """Demo: Retrieve a restaurant review."""
    print("\n" + "-" * 60)
    print("Demo: get_review('Iron & Embers')")
    print("-" * 60)

    data = await call_tool("get_review", {"restaurant_name": "Iron & Embers"}
    print(json.dumps(data, indent=2))
```

The review record includes the reviewer's name, rating, full review text, image description, and visit date—all the fields that were added during the earlier augmentation step.

Main entry point

Wiring it all together

Finally, add the `main` function and entry point that runs the verification and all three demos in sequence.

Note: in this section you will need to write the calls to each of the demo functions we just created. Locate the lines that are marked as `TODO` and replace the `...` in the code with your own code.

Hint: the calls to the 3 demo functions will look very similar to `await verify_connection`

python

```python
# Main Entry Point
async def main():
    """Run all demos sequentially."""
    await ... #TODO
    await ... #TODO
    await ... #TODO
    await verify_connection()

if __name__ == "__main__":
    asyncio.run(main())
```

`asyncio.run` creates an event loop, runs `main` to completion, and then tears it down. Each demo opens its own `ClientSession` (via `call_tool`), which means a fresh server subprocess is started for each call—straightforward to reason about and sufficient for a demonstration script.

Run the MCP client

Run the client:

```bash
python client.py
```

Run

If you see the results of each demo, that means our MCP client is working! The three demo sections will print the raw JSON responses returned by the server tools. At the end you will see the output of `verify_connection` and it should look something like this:

```ruby
--- START SCREENSHOT ---

Discovered 3 tools:
  - get restuarant_info: ...
  - reccomend_by_vibe: ...
  - get_review: ...

All required tools verified!

Discovered 1 resources:
  - culinary-map: ...

Configured 1 roots:
  - project: ...
--- END SCREENSHOT ---
```

Take a screenshot of the section in your terminal output from `START SCREENSHOT` to `END SCREENSHOT` and name it `M4L2_Build_Test_MCP_Client.jpg`. This screenshot will be used as one of the submission components later for your Final Project.

Conclusion

You have built a complete **MCP client** that connects to the Connoisseur server, declares filesystem roots, handles delegated LLM sampling requests, and calls all three server tools.

By completing this lab, you have:

- Connected to an MCP server over **stdio** using `ClientSession`
- Registered a **roots callback** to declare permitted filesystem paths
- Implemented a **sampling callback** that proxies LLM calls through the Anthropic API
- Called `get_restaurant_info`, `recommend_by_vibe`, and `get_review` via the MCP protocol

Together, the server and client form a complete MCP application. In the next lab, we'll build on this basic client-server app; we will enhance the demo functions for intelligent tool selection and create a full-fledged MCP application!

Author(s)

Abdul Fatir | Data Scientist @ IBM

Set up the environment

First, create a virtual environment to keep dependencies isolated.

```bash
pip install virtualenv
virtualenv .venv
source .venv/bin/activate
```

Run

Install the required libraries:

```bash
pip install fastmcp==3.1.0 anthropic==0.84.0 mcp==1.25.0
```

Run

Download the data files and the server from the previous lab:

```bash
# Unstructured California culinary map
curl -O https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/R1

# Structured restaurant data (JSON)
curl -O https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/7I

# Augmented user reviews (JSON)
curl -O https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/su

# MCP server from the previous lab
curl -O https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/RI
```

Run

Create the client file:

```bash
touch client.py
```

Run

Open client.py in IDE

← Build an MCP Client

MCP client foundation →